PIKVA · VOLUME 2

Frontend Technical Manual

Every file, import, component, page and dependency of the Pikva web app, and how it works with the backend, explained from first principles.

Codebase: **KIDSPICKUP/FRONTEND** (Next.js 16 Pages Router · React 19 · TypeScript · Tailwind CSS 4 · Bun)

State described: commit 7d18594, 16 October 2026

Companion: Volume 1, Backend Technical Manual (KIDSPICKUP/BACKEND)

Contents

Part A · Foundations

1. What the web app is for
2. Core ideas in plain English
3. The technology stack
4. Folder map
5. How every signed-in page works

Part B · File by file

6. Configuration files
7. Wrappers around every page
8. The server-side session layer (lib, server-only)
9. API routes: the Backend-for-Frontend
10. lib: helpers safe for the browser
11. components: shared UI
12. Pages, one by one

Part C · End-to-end workflows

13. Loading any signed-in page
14. Signing in and out
15. Admin onboards a family
16. Parent adds a pickup person; admin approves
17. Parent gets today's code; gate releases
18. Page ↔ endpoint map

Part D · Owning the code

19. Running the web app
20. Testing
21. Recipe: adding a page
22. Deploying
23. Troubleshooting
24. Glossary

Part A · Foundations

The ideas behind a modern web frontend, and how Pikva's is put together

1. What the web app is for

The web app is the screen people use. One site serves all three roles, each in its own area:

Area	Who	Pages
`/admin`	School administrators	Dashboard, Pupils (list, detail, add), Pickup People, Approvals, Pickup History, Security Alerts, Audit Log, Users (list, add).
`/parent`	Parents	Overview, My Pupils (list, detail), Pickup People (with today's code and QR), Pickup History, Settings.
`/gate`	Gate staff	Gate Verification: type or scan a code, then release or refuse.
`/account`	Admins and gate staff	Own details and password.
`/`, `/login`, `/forgot-password`, `/reset-password`	Anyone	Landing, sign-in and password reset.

The golden rule: **the web app decides nothing important**. Every rule (who may see what, whether a code is valid) is enforced by the backend. The web app checks forms to give faster, friendlier messages, but the backend always checks again.

2. Core ideas in plain English

HTML, CSS, JavaScript and TypeScript

A browser shows pages built from **HTML** (structure: headings, buttons, forms), styled by **CSS** (colours, layout) and made interactive by **JavaScript**. **TypeScript** is JavaScript with types: you write `name: string` and the TypeScript checker warns you before running if you pass a number. Pikva's files end in `.ts` (TypeScript) and `.tsx` (TypeScript with JSX, see below). Types disappear when the code is built; they exist to catch mistakes.

React: components, props, state and hooks

React builds a page from **components**: functions that return what should appear. They are written in **JSX**, HTML-like syntax inside TypeScript:

```
function Greeting({ name }: { name: string }) {     // a component; "name" is a prop
  const [count, setCount] = useState(0);              // state: a value React remembers
  return <button onClick={() => setCount(count + 1)}>Hello {name}, clicked {count}</button>;
}
```

- **Props** are inputs passed from a parent component (`<Greeting name="Ada" />`).
- **State** (`useState`) is data the component remembers between renders. Calling the setter re-draws the component.
- **Hooks** are functions starting with `use`: `useState`, `useEffect` (run code after the component appears, e.g. fetch data, start the camera, and clean up when it disappears), `useRef` (hold a reference to an element or a value without re-drawing), `useRouter` (current URL and navigation).
- `className` is JSX's name for HTML's `class` attribute.

Next.js and the Pages Router

Next.js is a framework around React that adds routing, a server and a build system. Pikva uses the **Pages Router**: the file path decides the URL.

File	URL
`src/pages/login.tsx`	`/login`
`src/pages/admin/index.tsx`	`/admin`
`src/pages/admin/pupils/[id].tsx`	`/admin/pupils/7` (`[id]` is a variable part)
`src/pages/api/auth/login.ts`	`/api/auth/login`, an **API route** (server code, returns JSON, no page)
`src/pages/api/proxy/[...path].ts`	`/api/proxy/anything/here` (`[...path]` catches all remaining parts)

Server-side rendering and getServerSideProps

A page file can export `getServerSideProps`. Next.js runs it **on the server** for every request, before the page is shown. It can check the session, call the backend and either return `{ props }` (data for the page) or `{ redirect }`. The page's HTML is then built on the server and sent to the browser (**server-side rendering**, SSR). The browser then loads the JavaScript and React "attaches" to the existing HTML so buttons work: this is **hydration**.

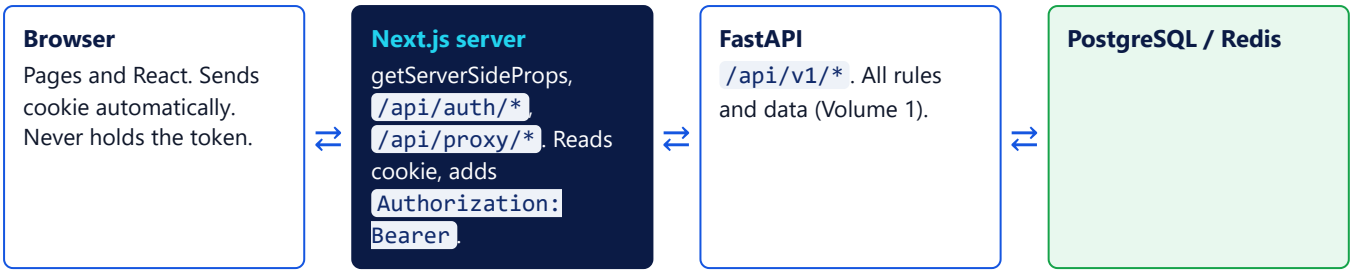
> **IN PLAIN ENGLISH**
>
> getServerSideProps is the waiter who checks your booking and fetches your order in the kitchen before you even sit down. The page component is the plate presentation. Hydration is the moment the cutlery arrives and you can interact.

Cookies and httpOnly

A **cookie** is a small value the server asks the browser to store and send back with every request to the same site. An **httpOnly** cookie cannot be read by JavaScript in the page, so even if an attacker managed to inject a script, it could not steal the session. **SameSite=Strict** means the browser never sends the cookie when another website triggers the request. **Secure** means HTTPS only.

The Backend-for-Frontend (BFF) pattern

Pikva's browser code **never talks to FastAPI and never sees the JWT**. It talks only to the Next.js server on the same site. The Next.js server holds the token in the httpOnly cookie and calls FastAPI on the browser's behalf. This middle layer is called a **Backend-for-Frontend**.



WHY IT IS BUILT THIS WAY

Tokens kept in browser storage can be stolen by any injected script. Keeping them in an httpOnly cookie, and letting only the Next.js server forward them, removes that risk. It also means the FastAPI address (`PIKVA_API_URL`) is never exposed to the browser.

Content Security Policy (CSP) and nonces

A **CSP** is a response header telling the browser which scripts and styles it may run. Pikva's says: run only this site's scripts, and only those tagged with this page view's random **nonce** (a number used once). An attacker who sneaks `<script>` into a page cannot know the nonce, so the browser refuses to run it.

Tailwind CSS

Instead of writing CSS files, components use small **utility classes** directly: `className="rounded-lg bg-brand-600 px-4 py-2 text-white"` means rounded corners, brand blue background, horizontal padding 4 units, vertical 2, white text. Prefixes apply at screen sizes: `sm:` ≥640px, `lg:` ≥1024px, so `hidden lg:flex` is "hidden on phones, shown from laptop width".

3. The technology stack

From `FRONTEND/package.json` (exact versions in `bun.lock`).

Package / tool	What it is	Use in Pikva
Bun 1.3 (`packageManager`)	A fast JavaScript runtime, package manager and test runner.	`bun install` , `bun dev` , `bun test` .
next 16.3.8	The React framework.	Routing, SSR, API routes, build, the request `proxy` hook.
react, **react-dom** 19.2.8	UI library and its browser renderer.	All components.
@zxing/browser 0.2.1	Camera-based barcode/QR reader for browsers.	Gate QR scanner (loaded only when scanning starts).
@zxing/library 0.23.0	The core barcode library.	Draws the parent's QR code (encoder) as SVG.

typescript 5 (dev)	Type checker.	`bunx tsc --noEmit`.
tailwindcss 4 + @tailwindcss/postcss (dev)	Utility CSS and its build plugin.	All styling; tokens in `globals.css`.
eslint 9 + eslint-config-next (dev)	Code linter with Next.js rules.	`bun run lint`.
@playwright/test 1.63 (dev)	Browser automation and test runner.	End-to-end tests in `e2e/`.
@types/bun, @types/node, @types/react, @types/react-dom (dev)	Type definitions.	Lets TypeScript understand those APIs.
Inter font (`next/font/google`)	Typeface.	Downloaded at build time and served by the site itself (no request to Google from the browser).

`scripts` in package.json: `dev` (development server), `build` / `start` (production), `lint`, `test:e2e` (Playwright). `ignoreScripts` / `trustedDependencies` list the only packages (sharp, unrs-resolver) allowed to run install scripts.

4. Folder map

```
FRONTEND/
├── package.json, bun.lock          # dependencies and scripts
├── next.config.ts                  # security headers, caching, Next options
├── tsconfig.json                   # TypeScript settings; "@/..." = src/...
├── postcss.config.mjs              # enables Tailwind
├── eslint.config.mjs               # lint rules
├── playwright.config.ts            # browser test settings
├── .env.example                    # PIKVA_API_URL (copy to .env.local)
├── public/                         # static files: favicon, images/pikva-mark.png
├── src/
│   ├── proxy.ts                    # runs before every page: nonce + CSP
│   ├── styles/globals.css          # Tailwind + design tokens (colours)
│   ├── lib/                        # logic without UI (28 files)
│   ├── components/                 # shared UI pieces (10 files)
│   └── pages/                      # one file per URL
│       ├── _app.tsx, _document.tsx # wrappers around every page
│       ├── api/auth/*, api/proxy/* # server-only BFF routes
│       ├── admin/*, parent/*, gate/
│       └── login, forgot-password, reset-password, account, index, use-mobile-app
├── tests/                          # unit tests (bun test)
└── e2e/                            # browser tests (Playwright)
```

Three kinds of code, kept apart

Kind	Files	Runs where

Server-only	`lib/config.ts`, `lib/backend.ts`, `lib/session.ts`, `lib/auth-guard.ts`, `pages/api/**`, every `getServerSideProps`	The Next.js server. May read `PIKVA_API_URL` and the cookie.
Pure helpers	Most of `lib/` ("Safe to import from pages")	Anywhere. Plain functions: validate forms, build URLs, choose messages. Easy to unit test.
UI	`components/`, page components	Server (first render) and browser (after hydration).

WHY IT IS BUILT THIS WAY

Keeping logic in small pure functions in `lib/` and keeping components thin means most behaviour is tested with fast unit tests that need no browser.

5. How every signed-in page works

Every protected page follows the same recipe. Learn it once and you can read any page:

```
export const getServerSideProps: GetServerSideProps<Props> = async (context) => {
  const guard = await requireUser(context, ["school_admin"]);   // 1. signed in with the right role?
  if (!("props" in guard)) return guard;                        //    no → redirect (login or own home)
  const { user } = await guard.props;
  try {
    const [pupils, people] = await Promise.all([                // 2. load data from FastAPI in parallel
      getAsUser<Pupil[]>(context, "pupils"),
      getAsUser<PickupPerson[]>(context, "pickers"),
    ]);
    return { props: { user, data: { pupils, people } } };       // 3. hand it to the page
  } catch {
    return { props: { user, data: null } };                     // 4. failure → page shows LoadError
  }
};

export default function Page({ user, data }: Props) {           // 5. draw it inside the role's shell
  return <AppShell user={user} title="...">
    {data ? <Content data={data} /> : <LoadError message="..." />}
  </AppShell>;
}
```

Actions (save, approve, release) happen later in the browser through `sendAsUser(path, method, body, ...)`, which calls `/api/proxy/<path>`; the Next.js proxy adds the token and forwards to FastAPI `/api/v1/<path>`.

Part B · File by file

Configuration, the server-side session layer, helpers, components and pages

6. Configuration files

next.config.ts

Imports `path` (Node) and the `NextConfig` type. `SECURITY_HEADERS` is sent with every response:

Header	Effect
`Content-Security-Policy: frame-ancestors 'none'` and `X-Frame-Options: DENY`	No other site may show Pikva inside a frame (stops "clickjacking").
`X-Content-Type-Options: nosniff`	The browser must trust the declared file type.
`Referrer-Policy: strict-origin-when-cross-origin`	Other sites only see "pikva's domain", never page paths or query strings (which could hold a reset token).
`Permissions-Policy: camera=(self), microphone=(), geolocation=()`	Only Pikva itself may use the camera (QR scanning); microphone and location are off.

Other options: `reactStrictMode: true` (extra development checks), `poweredByHeader: false` (do not advertise Next.js), `turbopack.root` (pin the project root), and an extra rule giving every `/api/*` response `Cache-Control: no-store` so signed-in data and pickup codes are never cached.

tsconfig.json

`strict: true` turns on all type safety checks; `noEmit` (Next.js compiles, TypeScript only checks); `jsx: react-jsx`; `moduleResolution: bundler`; and `paths: {"@/*": ["./src/*"]}`, which is why imports read `@/lib/session` instead of `../../lib/session`.

postcss.config.mjs, eslint.config.mjs, .env.example

PostCSS runs the Tailwind plugin over CSS. ESLint uses Next.js's "core web vitals" and TypeScript rule sets and ignores build output. `.env.example` has one setting, `PIKVA_API_URL=http://127.0.0.1:8000`, deliberately without the `NEXT_PUBLIC_` prefix so Next.js never sends it to the browser. Copy it to `.env.local`.

playwright.config.ts

Browser tests live in `e2e/` and end in `.e2e.ts` (so `bun test` ignores them); one worker, in order, because journeys build on each other; 60-second timeout; desktop Chrome; base URL from `PIKVA_WEB_URL` or

`http://localhost:3000`; traces and screenshots kept on failure; `globalSetup` / `globalTeardown` create and remove a throwaway school.

7. Wrappers around every page

src/proxy.ts: a fresh CSP for every page view

Next.js runs the exported `proxy(request)` before rendering pages. It makes a new nonce (`newNonce()`), builds the policy (`contentSecurityPolicy(nonce, isDev)`), adds both to the **request** headers (`x-nonce` and `Content-Security-Policy`, which Next.js reads to tag its own scripts) and sets the policy on the **response**. The exported `config.matcher` limits it to pages: not `/api`, static files, images or the favicon, and not link prefetches.

src/pages/_app.tsx

Wraps every page. Imports `globals.css` once for the whole site, loads the **Inter** font with `next/font/google` (self-hosted at build time) and exposes it as the CSS variable `--font-inter`, then renders the current page: `<Component {...pageProps} />`.

src/pages/_document.tsx

Customises the outer HTML (`<html lang="en">`, `<body>`). `getInitialProps` reads the `x-nonce` header set by `proxy.ts`; `render()` passes it to `<Head nonce>` and `<NextScript nonce>` so Next.js's own tags run under the strict policy.

src/styles/globals.css

`@import "tailwindcss"` loads Tailwind. The `@theme` block defines the **design tokens** taken from the UI design image: navy shades (sidebar), brand blues (buttons, links), cyan accent (parent portal), surface/card/line (backgrounds and borders), ink/ink-muted (text), and success/danger/warning pairs (badges). Tailwind turns each token into classes such as `bg-navy-900` or `text-danger-600`. Change a colour here and it changes everywhere.

> **WATCH OUT**
> The strict CSP blocks inline `style="..."` attributes in production. Use Tailwind classes instead (see how `Logo.tsx` maps sizes to classes for this reason).

8. The server-side session layer (lib, server-only)

lib/config.ts

`apiBaseUrl()` returns `PIKVA_API_URL` without a trailing slash, or throws a clear error if it is not set.

lib/roles.ts

`type Role = "school_admin" | "gate_staff" | "parent"` (exactly FastAPI's values), `LOGIN_PATH = "/login"`, and `HOME_PATH`: admin → `/admin`, gate staff → `/gate`, parent → `/parent`. Safe for browser code.

lib/session.ts: the cookie

Export	What it does
`SESSION_COOKIE`	The cookie name, `pikva_session`.
`JWT_SHAPE` (private)	A pattern for "three base64url parts separated by dots"; anything else is refused so nothing can smuggle extra cookie attributes.
`secureCookies()`	True in production (cookie sent over HTTPS only).
`sessionCookie(token, maxAge, secure)`	Builds the `Set-Cookie` text: `pikva_session=<jwt>; Path=/; Max-Age=...; HttpOnly; SameSite=Strict[; Secure]`.
`clearedSessionCookie(secure)`	Same cookie with `Max-Age=0`, which deletes it.
`sessionToken(req)`	Reads the token from the request, or `null`.
`secondsUntilExpiry(token)`	Decodes the JWT's middle part (no verification needed; FastAPI verifies) and returns seconds until `exp`, so the cookie expires with the token.

lib/backend.ts: talking to FastAPI from API routes

- `forwardedFor(req)`: builds `X-Forwarded-For` = any incoming chain + this request's peer address, so FastAPI's per-IP rate limits apply to the real visitor rather than to the web server.
- `relay(response, res)`: copies FastAPI's status and body back to the browser, passing on only `content-type` and `retry-after` headers.
- `postPublic(req, res, path, fields)`: for signed-out POSTs (password reset): POST only, picks only the named fields from the body, forwards with the IP, relays the answer.
- `unavailable(res)`: 502 "Pikva service is unavailable" when FastAPI cannot be reached. `methodNotAllowed(res, allowed)`: 405 with `Allow`.

lib/auth-guard.ts: protecting pages

- `type CurrentUser`: id, school_id, name, email, phone, role, school_name (the shape of FastAPI `GET /auth/me`).
- `requireUser(context, allowed)`: no cookie → redirect to `/login`. Calls FastAPI `/auth/me` with the token. 401 → delete the cookie and redirect to login. Other errors → throw (Next.js shows its error page). Wrong role → redirect to **that user's own** home page. Right role → `{ props: { user } }`.
- `class BackendError`: an error carrying FastAPI's status, so a page can turn a 404 into Next.js's "not found".
- `getAsUser<T>(context, path)`: GET `/api/v1/<path>` with the token and return the JSON, or throw `BackendError`.
- `redirectIfSignedIn(context)`: for `/login`: if already signed in, go home; anything uncertain just shows the page, so it can never loop.

lib/csp.ts

`newNonce()`: 16 random bytes from `crypto.getRandomValues`, base64. `contentSecurityPolicy(nonce, isDev)` joins these directives:

Directive	Meaning
`default-src 'self'`	By default load things only from this site.
`script-src 'self' 'nonce-…' 'strict-dynamic'`	Only nonce-tagged scripts and the scripts they load (page chunks, the QR scanner). Development adds `'unsafe-eval'` for React's debugging.
`style-src`	Nonce-tagged styles in production; `'unsafe-inline'` in development for the dev server.
`img-src 'self' https: data: blob:`	Photos may come from any https host.
`connect-src 'self'`	Browser fetches only to this site (never directly to FastAPI).
`media-src 'self' blob:`	The camera stream.
`object-src 'none'`, `base-uri 'self'`, `form-action 'self'`, `frame-ancestors 'none'`	Close other injection routes.
`upgrade-insecure-requests` (production)	Turn any http sub-request into https.

9. API routes: the Backend-for-Frontend

These run only on the Next.js server. The browser calls them with ordinary `fetch`, and the cookie travels automatically.

Route	Behaviour
POST `/api/auth/login`	Requires string email and password (else 422). Calls FastAPI `/auth/login` with `X-Forwarded-For`. Errors (401, 422, 429 with Retry-After) are relayed. On success reads `access_token`, works out its lifetime, sets the httpOnly cookie, and answers `{ok: true}`: **the browser never receives the token**.
POST `/api/auth/logout`	Deletes the cookie; 204. (A JWT cannot be revoked; it simply expires, and no browser script ever had it.)
GET / **PATCH** `/api/auth/me`	No cookie → 401 without calling FastAPI. GET forwards to `/auth/me`; PATCH forwards only `name` and `phone`. A FastAPI 401 also deletes the cookie.
POST `/api/auth/change-password`	Forwards current and new password with the token. On success FastAPI returns a new token, which **replaces** the cookie so this browser stays signed in while others are signed out. 401 clears the cookie.
POST `/api/auth/password-reset/request` and `/confirm`	`postPublic` with only `email`, or only `token` and `new_password`.

`/api/proxy/[...path]` (any method)	The general gateway. Each path segment must match `^[A-Za-z0-9_-]+$` (so `..` tricks cannot reach other URLs) and the first segment must not be `auth` (sign-in has its own routes so the token never leaks); otherwise 404. No cookie → 401. Forwards method, query string and JSON body to `/api/v1/<path>` with the Bearer token and `X-Forwarded-For`; FastAPI unreachable → 502; FastAPI 401 → cookie deleted; answer relayed.

IN PLAIN ENGLISH

When the gate page "releases a pupil", the browser posts to `/api/proxy/pickup/release`. The proxy turns that into `POST http://127.0.0.1:8000/api/v1/pickup/release` with `Authorization: Bearer <token from cookie>` and passes FastAPI's answer straight back.

10. lib: helpers safe for the browser

These are plain functions and types with no secrets. Most are covered by unit tests in `tests/`. A pattern repeats: form helpers return either `{ body }` (ready to send) or `{ error }` (a sentence for the user), and edit helpers send **only changed fields**.

Talking to the API from the browser

File	Contents
`parent-api.ts`	`sendAsUser<T>(path, "POST"|"PATCH", body, notFound, tooManyWhat)`: fetches `/api/proxy/<path>` and always returns `{ok: true, data}` or `{ok: false, message}`. Network failure → "Could not reach Pikva"; 401 → goes to `/login`; 429 → "Too many … Try again in N minutes"; other errors via `actionErrorMessage` (404 → the caller's notFound text, 400/422 → FastAPI's sentence, 403, 409, else "unavailable"). Every page action uses this.
`api-errors.ts`	`UNAVAILABLE`, `OFFLINE` messages; `detailText(body)` extracts FastAPI's `detail` when it is a sentence; `tooManyMessage(what, retryAfter)`.
`login-error.ts`, `reset-error.ts`, `change-password.ts`	Turn status codes into messages for those forms; check new passwords (1–128 characters, not all spaces, matching confirmation, different from current); `isResetTokenShape`.
`sign-out.ts`	`signOut()`: POST `/api/auth/logout`, then a **full** page load of `/login` so no signed-in data stays in memory.

Data shapes and domain helpers

File	Contents
`parent.ts`	Types matching FastAPI: `Pupil`, `PickupPerson`, `AuthorisationStatus`, `Release`. Helpers: `pupilName`; `isSameLocalDay`, `todaysReleases`, `latestReleaseTodayByPupil` (today in the **viewer's** time zone); `peopleCountsByPupil`, `peopleCountsText`; `formatTime`, `formatDateTime` (viewer's locale), `formatDate` (fixed en-GB/UTC so server and browser agree); `isWebPhoto` (only http(s) links are shown as images); pupil form helpers `pupilForm`, `pupilFormError`, `pupilChanges`; `localIsoDate`; history paging `HISTORY_PAGE_SIZE` (25), `parsePupilFilter`, `historyApiPath` (asks for 26 rows: a 26th means "there is a next page", since FastAPI sends no total), `historyHref`, `groupReleasesByDay` ("Today", "Yesterday", dates).
`pickup-people.ts`	The add/edit form for pickup people: `toLocalInput` / `fromLocalInput` convert between the browser's date-time input (local time) and UTC ISO text for FastAPI; `emptyPersonForm`, `personForm`; `personBody(form, now, original?)` validates (lengths match the backend; end after start; not already past) and returns a full POST body or only changed fields for PATCH; `changeMessage` adds "Your school needs to approve them" when the result is pending; `PickupCode` type; `formatCode` ("7F3B 9K2D").

`gate.ts`	Types for verify/refuse/release answers and all `CheckResult` names; `normaliseCode` (strip spaces/dashes, upper-case, check the 32-character alphabet and length 8 **before** sending, because every failed check at the backend raises a security alert); `FAILURE_TEXT` / `failureText` (what staff are told for each result); `parseQrData` (accept only Pikva's `{"code": ...}` JSON, max 512 chars, so scanning a random QR never raises an alert); `cameraErrorText` (blocked, no camera, camera busy).
`qr.ts`	`encodeQr(text)` loads `@zxing/library` only when needed, encodes a QR with a 4-module quiet zone, and `qrSvgPath` turns the dark squares into one SVG path (no image files, works with the CSP).
`users.ts`	`SchoolUser` type; role filters; `parsePage`, `parseRoleFilter`; `usersApiPath` (51 rows for a 50-row page); `usersPageHref`; `CREATABLE_ROLES` (parent, gate staff only); `newUserBody`, `userChanges`, `isManageable`.
`admin-pupils.ts`	`searchPupils` (every word must appear in pupil or parent name; `fold` removes accents and case so "chidi" finds "Chídí"); `PARENTS_API_PATH`; `newPupilBody`; `filterParents`.
`admin-pickup-people.ts`	`STATUS_FILTERS`, `parseStatusFilter`, `personRows` (join people to pupils and sort), `filterPersonRows`, `statusCounts`, `peopleHref`.
`approvals.ts`	`pendingApprovals` (pending people, longest-waiting first, with their pupil); `approveBody` sends `updated_at` back for the backend's version check.
`alerts.ts`	`SecurityAlert` type; filters open/resolved/all; paging; `alertTitle` (friendly names); `isUrgent` (repeated attempts, refused pickup); `BELL_LIMIT` (100) and `bellBadge` ("99+").
`audit.ts`	`AuditRecord` type; `ACTION_LABELS` (every backend action in plain words, e.g. `pupil.released` → "Released a pupil"); filters and paging; `auditNotes` (turns metadata into notes like "Changed: name, phone", "Needs school approval again", "QR code"); `auditLinks`.
`dashboard.ts`	`DASHBOARD_LIMIT` (200 rows read), `PANEL_SIZE` (5), `countText` ("200+" when the list was full), `mayHaveMore`.
`history-dates.ts`	The day filter: `parseDay` (real calendar date), `parseTimeZone` (a zone the server knows, else UTC), `dayRange` (that day's local midnight to next midnight as UTC `since` / `until`, correct across clock changes), `dayLabel`.
`navigation.ts`	The menu for each role (`MENUS`), `ROLE_LABEL`, `navItems`, `isActive` (home items match exactly; others also match sub-pages).
`own-details.ts`	`ownDetailsChanges`: name/phone validation and changed fields only.
`format.ts`	`initials` ("Ada Okafor" → "AO"), `firstName`.
`use-now.ts`	The `useNow()` hook: `null` during the server render and hydration, the mount time in the browser. Pages show "today" and clock times only once it is set.

WHY IT IS BUILT THIS WAY

Why `useNow`? The server does not know the visitor's time zone, and the backend has no school time zone. If the server rendered "10:24" in UTC and the browser re-rendered "11:24" in Lagos, React would report a hydration mismatch. So times wait for the browser, showing a grey skeleton for an instant.

11. components: shared UI

Component	What it renders and does
`AppShell`	Layout for admins and gate staff: navy sidebar with the role's menu (`navItems`) and a school card; top bar with the admin `AlertBell`, the user's name (links to `/account`) and Sign out; the page title and content. Below laptop width the sidebar slides in from a menu button and closes after navigating (`router.events`). Sets the tab title.
`AlertBell` (inside AppShell)	After the page loads, fetches `/api/proxy/alerts?status=open&limit=100` and shows the count badge. Cancelled if you leave (`AbortController`); failures just show no number.
`ParentShell`	Lighter layout for parents: white sidebar with an avatar card on large screens; on phones a compact top bar and a bottom tab bar with short labels.
`AuthCard`, `FormError`	The centred card used by sign-in and password pages; red error box (`role="alert"` so screen readers announce it). Also exports shared class strings `inputClass`, `primaryButtonClass`, `linkClass`.
`ParentUi`	Small blocks: `LoadError` (with "Try again" re-running getServerSideProps via `router.replace`), `EmptyState`, `Skeleton`, `PupilAvatar` (photo or initials; photos load with `referrerPolicy="no-referrer"`), `StatusBadge` (colour per status), `Field` (label + input), `SuccessNote`.
`PupilPanels`	Shared by parent and admin pupil pages: `PupilDetails` (view/edit with `pupilChanges` → PATCH `pupils/{id}`), `PickupPeopleList`, `RecentPickups` (`showStaff` only on admin pages).
`OwnDetailsCard`	Your name/phone/email with an edit form → PATCH `/api/auth/me`; on success reloads page data so the new name shows in the top bar.
`ChangePasswordForm`	Current/new/confirm → `/api/auth/change-password`; explains that other devices are signed out.
`DayField`	A date input for history pages plus a hidden `tz` field filled with the browser's time zone, so the server can work out "that day" for the viewer.
`Icon`, `Logo`	Inline SVG outline icons by name; the Pikva shield image (`/images/pikva-mark.png`) and wordmark, sized with classes because inline styles are blocked.

12. Pages, one by one

For each page: who may open it (the roles given to `requireUser`), what it loads on the server, and what it does in the browser. Endpoint paths are FastAPI's, without `/api/v1`.

Public and account pages

Page	Details
`/` index.tsx	Landing with the logo and a Sign In button. Has an empty getServerSideProps only so each view gets a CSP nonce.
`/login`	`redirectIfSignedIn`. Form → `/api/auth/login`; then `/api/auth/me` for the role; `router.replace(HOME_PATH[role])`. Show/Hide password; errors via `loginErrorMessage`; link to forgot password.
`/forgot-password`	Email → `/api/auth/password-reset/request`; always shows the same "if an account exists…" message.
`/reset-password?token=`	Server checks the token's shape. Form → `/api/auth/password-reset/confirm`. States: form, "Password changed" (link to sign in), "Link not valid" (link to request a new one). A `no-referrer` meta tag keeps the token out of other sites' logs.
`/account`	Admin, gate staff. `OwnDetailsCard` (plus Role and School rows) and `ChangePasswordForm` in `AppShell`.
`/use-mobile-app`	Old link; always redirects to `/parent`.

Gate: /gate (gate staff)

A small **state machine** held in `step`: `enter` → `verified` → `released` / `refused` / `failed` → back to `enter` with "Next pickup".

- `CodeEntry`: large code box focused automatically; `normaliseCode` catches typos locally; "Check code" → `sendAsUser("pickup/verify", {method: "manual", code})`. "Scan QR code" opens `QrScanner`.
- `QrScanner`: requires a secure context; dynamically imports `@zxing/browser`; `BrowserQRCodeReader.decodeFromConstraints({video: {facingMode: "environment"}})` uses the back camera; on the first result stops the camera and passes the text through `parseQrData`, then verifies with `method: "qr"`. Cleanup stops the camera when the component goes away. Camera errors offer typing instead.
- `ConfirmRelease`: green "Code valid" card with pupil and pickup person photos/names (`Person`; "No photo — check their identity another way"). "Release <name>" → `pickup/release`. "Don't release" asks for confirmation, then `pickup/refuse`. Both resend the same request the check used.
- `Failed`, `Refused`, `Released`: clear red/green result cards; `NextButton` gets focus so Enter starts the next pickup.

Parent portal (parent)

Page	Loads · Does
`/parent` Overview	`pupils`, `pickers`, `releases?limit=50`. Stats (pupils, active pickup people, picked up today), a card per pupil (today's status, people counts), today's pickups list.
`/parent/pupils`	`pupils`, `pickers`. List with links to each pupil.
`/parent/pupils/[id]`	`pupils/{id}`, `pickers?pupil_id=`, `releases?pupil_id=&limit=10`; 404 → not found page. `PupilDetails` (editable), people, recent pickups.

`/parent/pickup-people`	`pupils`, `pickers`. `PickupPeopleManager` keeps the list in state; `PersonEditor` adds (POST `pickers`) or edits (PATCH `pickers/{id}`); `PersonCard` activates/deactivates; pending people show a "waiting for your school" note; **active** people get `TodaysCode`.
`/parent/pickup-history`	Filters: pupil, day (`DayField` + `dayRange`), page. Grouped by day.
`/parent/settings`	Own details and password.

TodaysCode (in pickup-people) has steps `idle` → `confirm` ("any earlier unused code will stop working") → POST `pickup/codes` → `shown`: the code in large letters, its expiry, the QR drawn by `QrImage` from `qr_data`, Copy and Hide. The code lives only in this component's memory: never stored, never in the URL; Hide forgets it.

Admin area (school_admin)

Page	Loads · Does
`/admin` Dashboard	`pupils`, `pickers`, `releases?limit=200`, `alerts?status=open&limit=200`. Stats (pupils, active people, picked up today, open alerts with urgent highlight), panels with links: waiting approvals, today's pickups, open alerts.
`/admin/pupils`	`pupils`, `pickers`; search `?q=` done on the page (`searchPupils`) since FastAPI has no search.
`/admin/pupils/new`	Parents list (`schools/users?role=parent&limit=500`), filterable picker; POST `pupils` with `parent_id`; then goes to the pupil page with `?added=1`.
`/admin/pupils/[id]`	Like the parent page plus a `ParentCard` (contact details) and staff names on recent pickups.
`/admin/pickup-people`	`pickers`, `pupils`; status filter and search on the page; per row Edit (`AdminPersonEditor`, PATCH) and Activate/Deactivate. Admins never create pickup people.
`/admin/approvals`	Pending people (`pendingApprovals`) with photo, details and requesting parent. Approve → POST `pickers/{id}/approve` with `updated_at`; Decline → `deactivate` after confirming. A 409 shows FastAPI's reason.
`/admin/pickup-history`	Same filters as the parent version, plus who released and how.
`/admin/alerts`	Alerts page plus pupils, pickers and gate staff to turn ids into names; filter open/resolved/all; paging; urgent alerts highlighted; Resolve → POST `alerts/{id}/resolve`.
`/admin/audit-log`	Audit records with action filter and paging; plain-language labels and notes; links to pupils/people. Read-only.
`/admin/users`	`schools/users` with role filter and 50-row pages; Edit (PATCH `schools/users/{id}`) and Deactivate/Activate for parents and gate staff (never admins).
`/admin/users/new`	Name, email, phone, role (parent or gate staff) → POST `schools/users`; confirmation explains the set-password email; "Add another".

Part C · End-to-end workflows

What happens in the browser, the Next.js server and FastAPI, step by step

Legend: User Browser Next.js FastAPI DB . Backend internals of each step are in Volume 1, Part C.

13. Loading any signed-in page

1. Browser requests `/admin/pupils`, sending the `pikva_session` cookie.
2. proxy.ts creates a nonce and CSP for this view.
3. getServerSideProps → `requireUser` → FastAPI `GET /auth/me` with the Bearer token. No cookie or 401 → redirect to `/login` (cookie deleted). Wrong role → redirect to the user's own home.
4. getAsUser loads `pupils` and `pickers` in parallel from FastAPI, which filters them to the admin's school.
5. Next.js renders the page HTML with the data and the nonce-tagged scripts; response is `no-store`.
6. Browser shows the HTML, loads JavaScript, React hydrates; `useNow` then fills in local times; the admin bell fetches the open alert count.

14. Signing in and out

1. User submits email + password on `/login`.
2. Browser `POST /api/auth/login` (same site).
3. Next.js → FastAPI `POST /api/v1/auth/login` with `X-Forwarded-For`. Rate limits and password check happen there.
4. Next.js receives the JWT, sets the httpOnly cookie (lifetime = token's), returns `{ok:true}`.
5. Browser `GET /api/auth/me` → role → `router.replace("/admin" | "/gate" | "/parent")`.
6. **Sign out**: `POST /api/auth/logout` deletes the cookie, then a full reload of `/login`.
7. **Session ends on its own** (30 minutes, password changed elsewhere, account deactivated): the next FastAPI call answers 401; Next.js deletes the cookie; pages redirect to login and `sendAsUser` sends the browser there.

15. Admin onboards a family

1. `/admin/users/new`: add the parent (POST `schools/users`). FastAPI emails them a set-password link.
2. The parent opens `/reset-password?token=…`, chooses a password, then signs in.
3. `/admin/pupils/new`: choose that parent, enter the pupil, POST `pupils`; land on the pupil page with a confirmation.

16. Parent adds a pickup person; admin approves

1. **Parent** on `/parent/pickup-people` → "Add pickup person" → form; `personBody` validates and converts local times to UTC.
2. **Browser** `sendAsUser("pickers", "POST", body)` → `/api/proxy/pickers` → FastAPI. The new card shows "Awaiting approval" and the note that the school must approve.
3. **FastAPI** emails the school's admins (cooldown permitting).
4. **Admin** sees the dashboard panel and the Approvals page; clicks Approve → `pickers/{id}/approve` with the `updated_at` the page loaded. If the parent changed the person meanwhile, a 409 says to review again.

17. Parent gets today's code; gate releases

1. **Parent** on an **active** person's card: "Get today's code" → confirm.
2. **Browser** `sendAsUser("pickup/codes", "POST", {authorisation_id})`.
3. **FastAPI** 201 with `code` and `qr_data`; **Browser** shows "7F3B 9K2D" and draws the QR with `encodeQr`. The parent copies or screenshots it for the pickup person.
4. **Gate staff** on `/gate` type the code (checked by `normaliseCode`) or scan the QR (checked by `parseQrData`).
5. **Browser** → `pickup/verify`. Valid: green card with photos. Invalid: red "Do not release" card with the reason; an alert has been recorded.
6. **Gate staff** compare the person with the photo, then **Release** (`pickup/release` → 201, "<name> released at 3:12 PM") or **Refuse** (`pickup/refuse`: code cancelled, school alerted).
7. Both the parent's Overview/History and the admin's Dashboard/History now show the release; parents never see which staff member released.

18. Page ↔ endpoint map

Page	FastAPI endpoints used

login, forgot/reset password	`auth/login`, `auth/me`, `auth/password-reset/request`, `/confirm`
account, parent/settings	`auth/me` (GET, PATCH), `auth/change-password`
gate	`pickup/verify`, `pickup/release`, `pickup/refuse`
parent pages	`pupils`, `pupils/{id}` (GET, PATCH), `pickers` (GET, POST), `pickers/{id}` (PATCH), `pickers/{id}/activate|deactivate`, `pickup/codes`, `releases`
admin dashboard	`pupils`, `pickers`, `releases`, `alerts`
admin pupils	`pupils` (GET, POST, PATCH), `schools/users?role=parent`, `pickers?pupil_id`, `releases?pupil_id`
admin pickup people, approvals	`pickers`, `pupils`, `pickers/{id}` (PATCH), `activate|deactivate|approve`
admin history, alerts, audit	`releases`, `alerts`, `alerts/{id}/resolve`, `audit-logs`, `schools/users?role=gate_staff`
admin users	`schools/users` (GET, POST), `schools/users/{id}` (PATCH), `/activate|deactivate`

Part D · Owning the code

Run, test, change, deploy

19. Running the web app

```
# the backend must be running first (Volume 1, Part D)
cd FRONTEND
cp .env.example .env.local        # PIKVA_API_URL=http://127.0.0.1:8000
bun install
bun dev                           # http://localhost:3000
bunx tsc --noEmit                 # type check
bun run lint                      # lint
```

20. Testing

Unit tests: bun test (tests/)

Fast tests of the pure helpers (`gate.test.ts`, `parent.test.ts`, `pickup-people.test.ts`, `history-dates.test.ts`, `csp.test.ts`, ...) and of the API routes. `tests/helpers.ts` starts a **fake FastAPI** with `Bun.serve` on a free port that records every request and replies as the test decides, plus minimal fake Next.js request/response objects, so tests like `proxy-route.test.ts` and `auth-routes.test.ts` can prove, for example, that the token is never returned to the browser and that `../` paths are refused.

Browser tests: bun run test:e2e (e2e/)

Needs FastAPI and `bun dev` running and Chromium installed once (`bunx playwright install chromium`). `global-setup.ts` runs `BACKEND/scripts/e2e_school.py create` to make a tagged throwaway school and signs each role in, saving browser state; `global-teardown.ts` deletes that school. Files:

- `pickup-journey.e2e.ts`: the full story in a real browser: parent adds a person → admin approves → parent gets a code → gate verifies and releases → both histories show it.
- `admin.e2e.ts`, `settings.e2e.ts`: admin pages and account settings.
- `layout.e2e.ts`: every page at phone and laptop width (no horizontal overflow).

21. Recipe: adding a page

1. Create `src/pages/admin/something.tsx` (the URL follows the file).

2. Write `getServerSideProps` with `requireUser(context, ["school_admin"])` and `getAsUser` calls; catch errors and pass `data: null`.
3. Put types and logic (parsing query parameters, building API paths, validating forms) in a new `src/lib/something.ts` and unit-test it.
4. Render inside `AppShell` (or `ParentShell`); show `LoadError` when data is null; use `sendAsUser` for actions; style with Tailwind tokens, no inline styles.
5. Add a menu entry in `lib/navigation.ts` only once the page exists.
6. Check: type check, lint, unit tests, and the layout e2e at phone width.

> **WATCH OUT**
>
> Never put secrets or the FastAPI address in a `NEXT_PUBLIC_` variable, never store the token or a pickup code in `localStorage` or the URL, and never trust a check done only in the browser: the backend must enforce every rule.

22. Deploying

Follow `FRONTEND/DEPLOYMENT.md`: set `PIKVA_API_URL` on the server, `bun install --frozen-lockfile`, `bun run build`, `bun run start` behind HTTPS (the cookie is `Secure` in production, and the camera only works on https). Make sure the backend's `FORWARDED_ALLOW_IPS` trusts the web host and its `WEB_APP_URL` matches this site's address.

23. Troubleshooting

Symptom	Likely cause
Error "PIKVA_API_URL is not set"	Missing `.env.local`; restart `bun dev` after creating it.
"Pikva service is unavailable" (502)	FastAPI not running or wrong URL.
Signed out unexpectedly	30-minute token expired, password changed on another device, or account deactivated.
Camera does not start at the gate	Not on https or localhost, permission denied, or camera in use. Typing always works.
Times show as grey bars briefly	Normal: they wait for the browser's time zone (`useNow`).
Styles or scripts blocked (console mentions CSP)	Inline `style=` or an external script; use classes and same-site code.
"Get today's code" missing	Only active pickup people (approved, within dates) can get codes.

24. Glossary

API route

A file under `pages/api` that runs on the Next.js server and returns data, not a page.

BFF (Backend-for-Frontend)
A server layer owned by the web app that talks to the real backend for it.

Component
A function returning a piece of UI.

CSP / nonce
A header limiting what may run / a random per-view value that marks allowed scripts.

getServerSideProps
Server code that runs before a page renders: guard, load data, redirect.

Hook
A React function starting with `use` that gives a component state or effects.

Hydration
React attaching interactivity to server-rendered HTML.

httpOnly cookie
A cookie page scripts cannot read.

JSX
HTML-like syntax inside TypeScript describing UI.

Props / state
Inputs to a component / values it remembers.

SSR
Building a page's HTML on the server.

Tailwind utility class
A small class doing one styling job, like `px-4`.

TypeScript
JavaScript with types checked before running.